

Binder Generation with BindCraft

Table of Contents

Table of Contents	1
1. Prerequisites	2
1.1 Install VS Code.....	2
1.2 Github Account	2
1.3 La Trobe VPN	2
1.4 Remote - SSH extension	2
2. Connect to the Ewok Machine	3
3. Binders Generation with BindCraft	5
3.1 Open the BindCraft Project	5
3.2 BindCraft UI	6
3.2.1 Set Up the Notebook	6
3.2.2 Run a bindCraft job via the UI	7
3.3 Licensing.....	13
4. Useful Extensions	14
Codex.....	14
Office Viewer	14
Protein Viewer	15

1. Prerequisites

1.1 Install VS Code

Download and install VS Code from the official download page:

https://code.visualstudio.com/download?exp_download=fb315fc982

1.2 Github Account

Create a GitHub account at <https://github.com/> if you do not already have one.

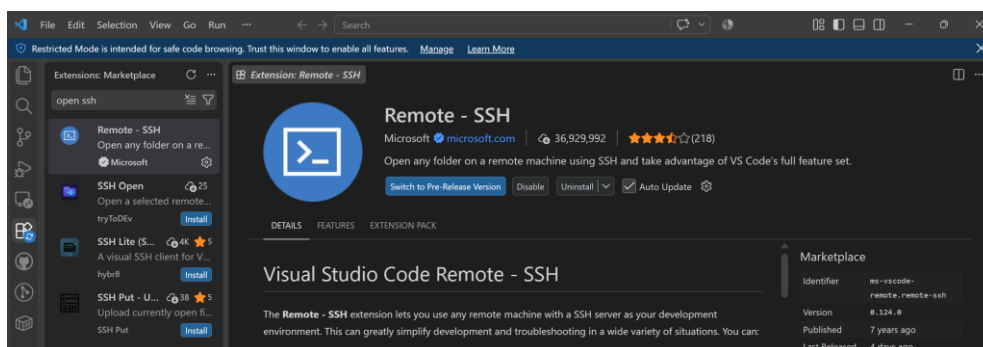
1.3 La Trobe VPN

To securely connect to Ewok machine off campus, download and setup the VPN here:

<https://www.latrobe.edu.au/students/support/it/connecting/vpn>

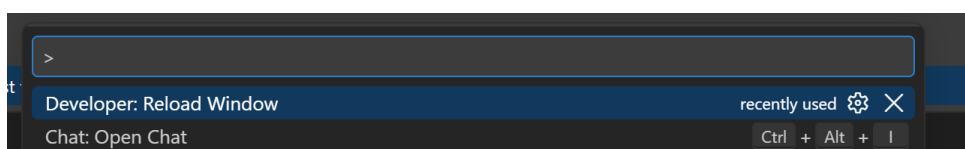
1.4 Remote- SSH extension

Install the **Remote – SSH** extension in VS Code. This extension enables you to connect to an Ewok machine from your local computer and access its computational resources directly through VS Code.



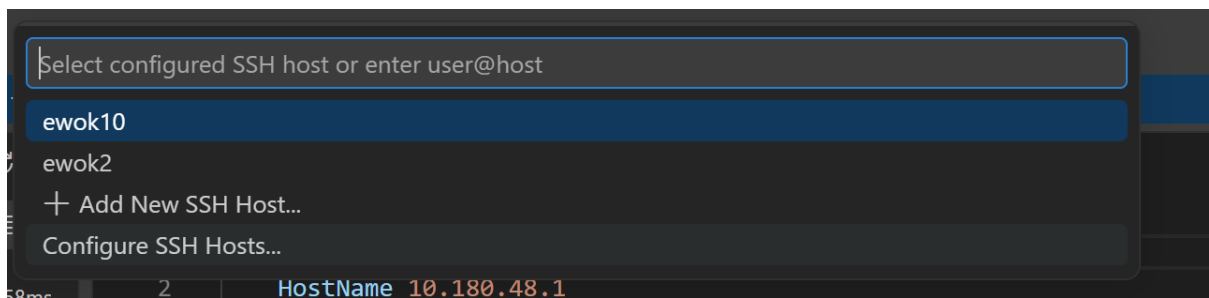
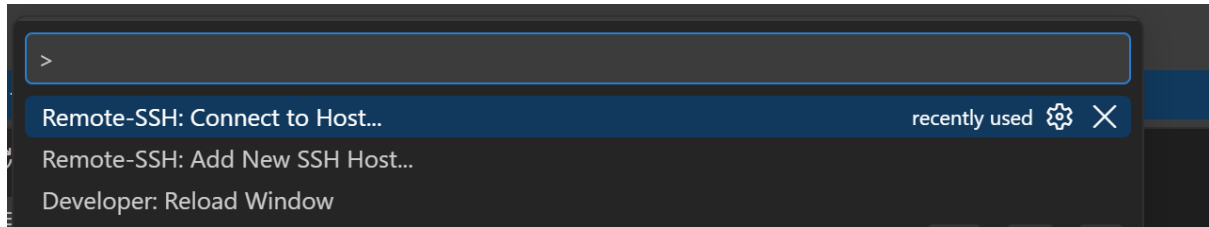
After installation, open the **Command Palette** (**Ctrl + Shift + P** on Windows or **Cmd + Shift + P** on Mac.) and type **>Developer: Reload Window**, and press Enter to ensure the extension loads correctly.

Quick tip: To run any command in the **Command Palette**, make sure to start it with a **>** symbol, e.g. **>your command**.



2. Connect to the Ewok Machine

In VS Code, open the **Command Palette** and type **>Remote-SSH: Connect to Host**, then select **Configure SSH Hosts**.



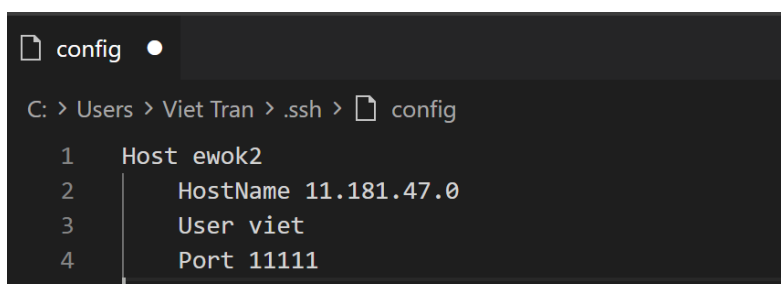
Add an entry for your Ewok machine using the following configuration format:

```
Host <ewok_machine_number>
  HostName <your_ewok_machine_ip_address>
  User <your_ewok_username>
  Port <your_ewok_port>
```

Example configuration:

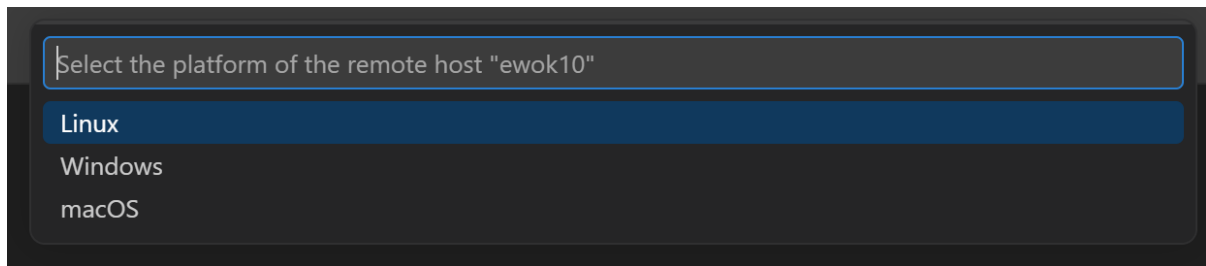
```
Host ewok2
  HostName 11.181.47.0
  User viet
  Port 11111
```

Make sure to save your changes to the config via **Ctrl + S** on Windows or **Cmd + S** on Mac.

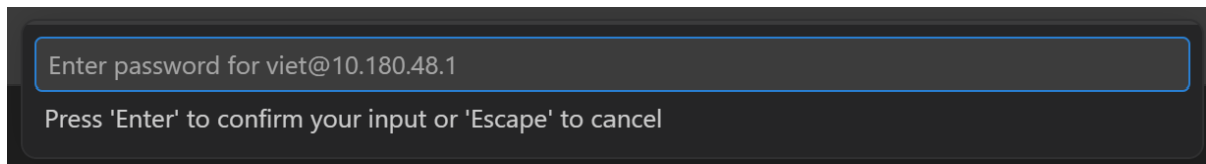


Note: Contact [Zhen He](#) or [Aaron Harris](#) for setting up and access your Ewok account.

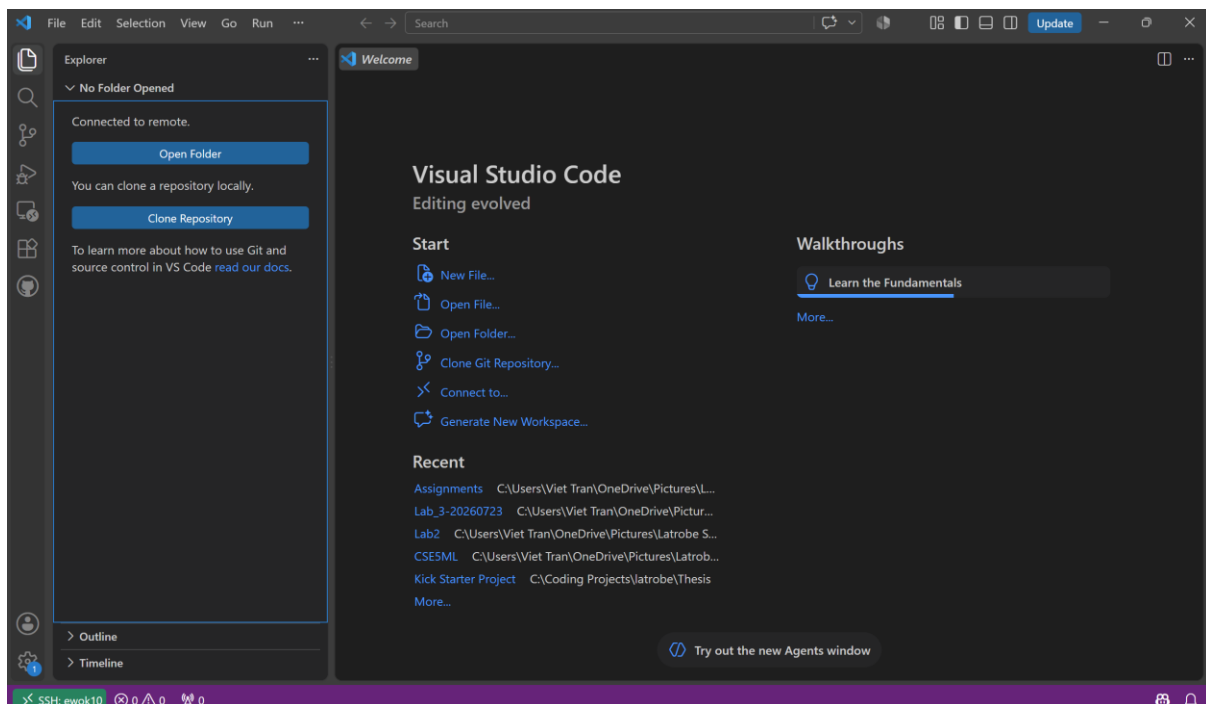
If VS Code prompts you to select the remote host's operating system, choose **Linux**



Enter your Ewok password to complete the connection.



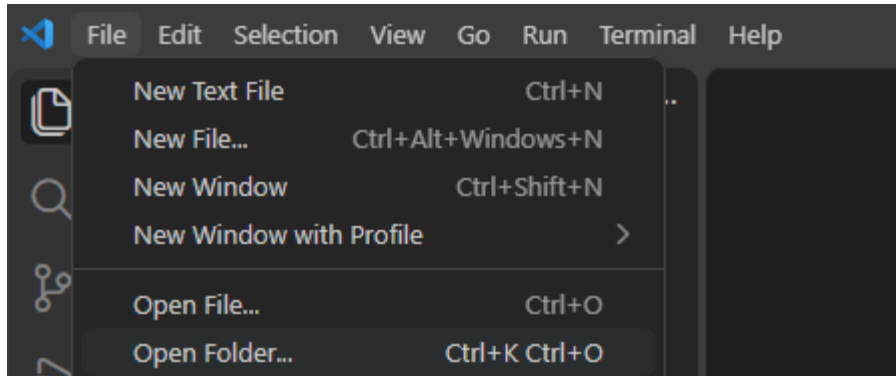
Once the connection is established, the Ewok host name should appear in the bottom-left corner of the VS Code window.



3. Binders Generation with BindCraft

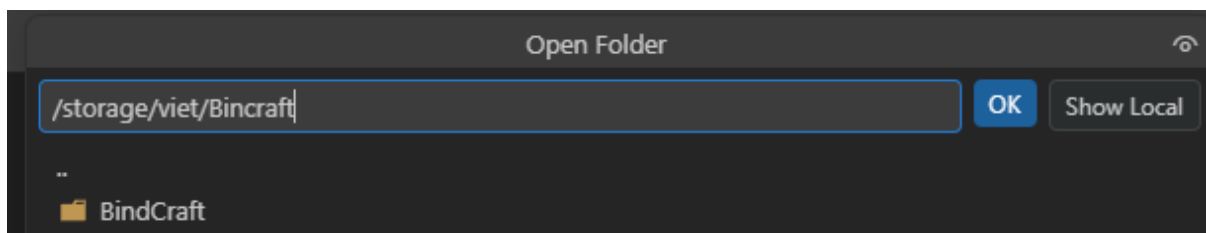
3.1 Open the BindCraft Project

After connecting to the Ewok machine, select **File** → **Open Folder** in VS Code.



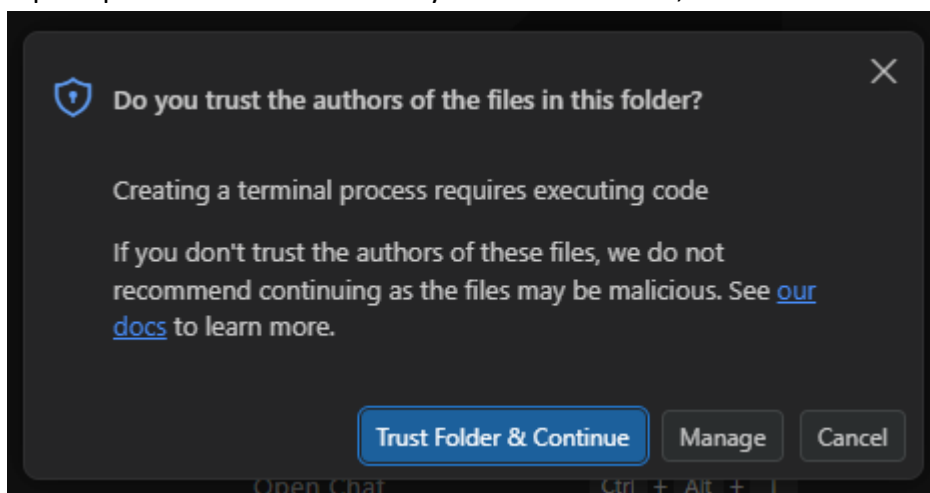
Open the directory in which BindCraft is installed. Typical installation locations include:

- /home/<username>/BindCraft
- /storage/<username>/BindCraft



Replace <username> with your Ewok username.

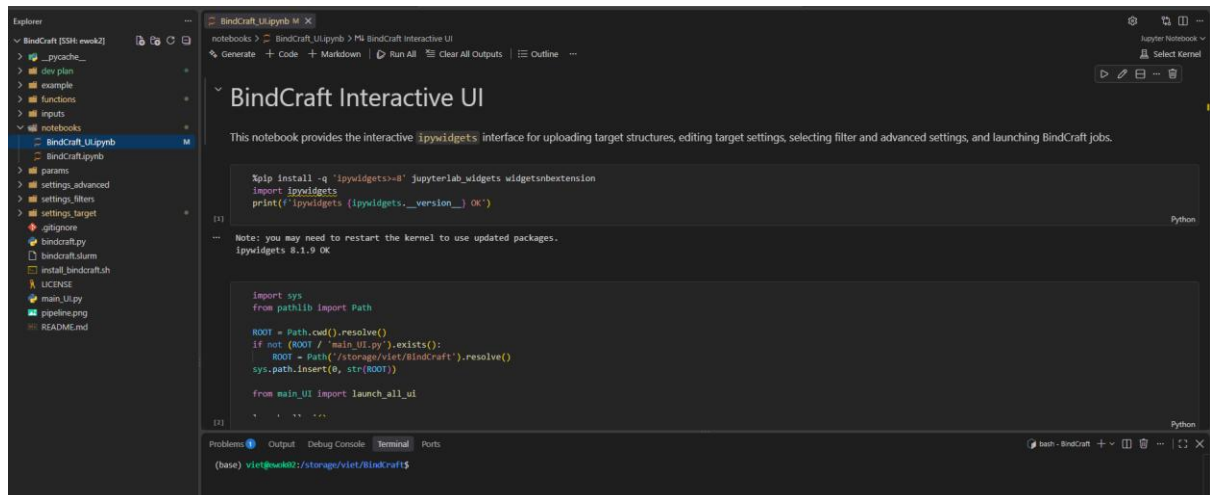
If prompted to confirm whether you trust the folder, select **Trust Folder & Continue**.



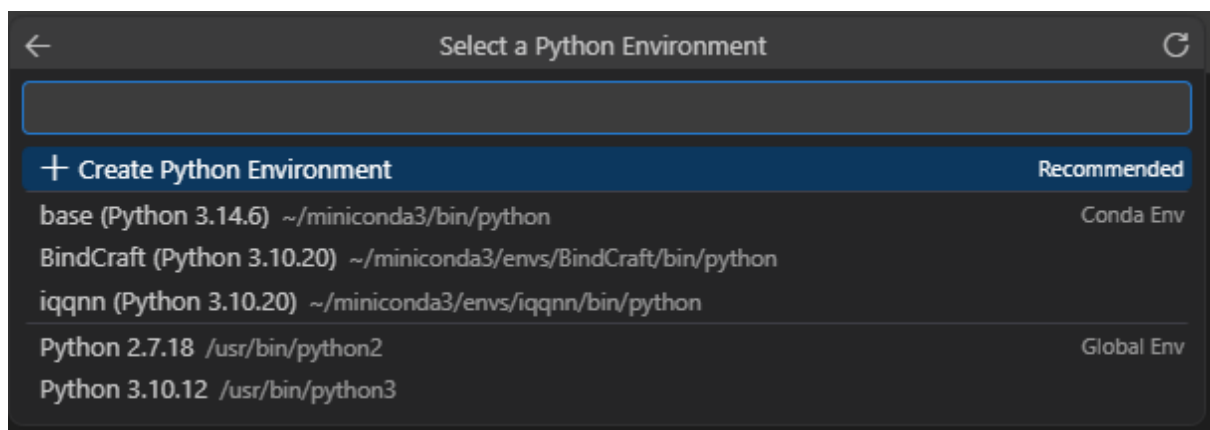
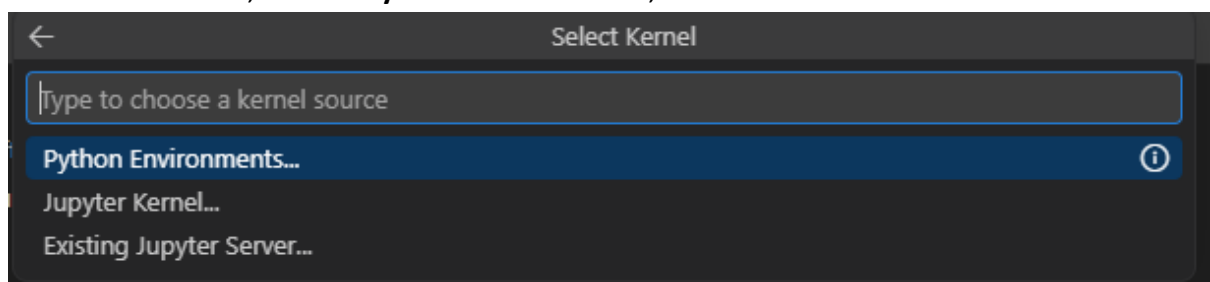
3.2 BindCraft UI

3.2.1 Set Up the Notebook

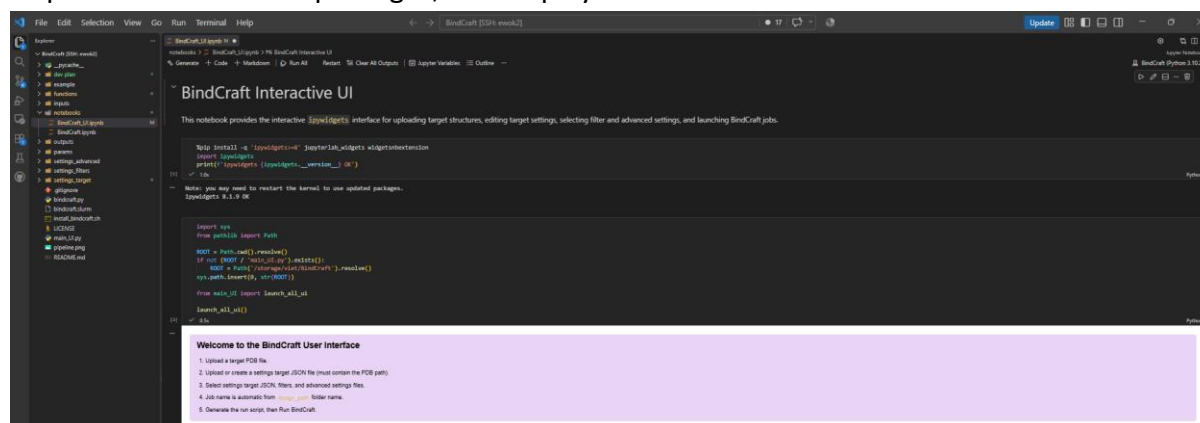
In the **Explorer** panel on the left side of VS Code, open notebooks/BindCraft_UI.ipynb



Click **Select Kernel**, choose **Python Environments**, and select the **BindCraft** environment.



After kernel successfully connected, click **Run All** to execute the notebook cells, install the required user interface packages, and display the interactive BindCraft interface.



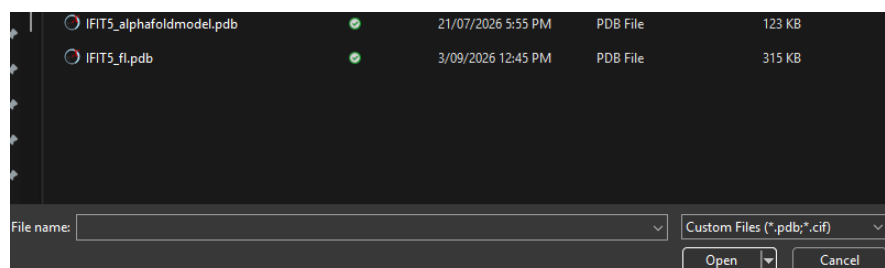
3.2.2 Run a bindCraft job via the UI

Step 1: Upload a Target PDB file

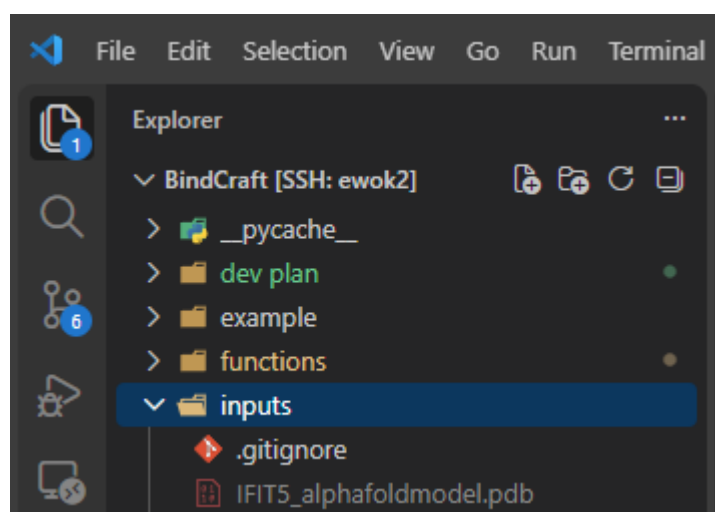
Step 1: If not already present, upload a PDB file to /storage/viet/BindCraft/inputs/

Upload PDB (0)

Use the **Upload PDB** control to select and upload the target PDB file from your local computer.



The uploaded file will be saved in the inputs/ directory within the BindCraft project:



Step 2: Configure the Target Settings

The target settings JSON file specifies the target structure against which BindCraft will generate binders. It also defines the binder naming prefix, output directory, target chains, hotspot residues, binder length range, and requested number of accepted designs.

If you do not already have a target settings JSON file, create one directly through the user interface by completing the following fields:

Field	Description
File Name	Target settings JSON filename.
design_path	Directory for outputs of a BindCraft run.
binder_name	The prefix used for generated binder filenames, for example, IFIT5_bindcraft_ewok-Binder.
starting_pdb	The uploaded target PDB file.
chains	The target chains to include, for example, A or A,C. Other chains will be ignored.
hotspots	The target residues intended for binder interaction, for example, A56,A60-65. Leave this field empty for automatic selection.
lengths	The minimum and maximum binder lengths, for example, [65,150].
Number designs	The requested number of accepted binder designs.

Or you can edit/create a new target JSON file.

File Name:

design_path:

binder_name:

starting_pdb: ▼

 Refresh PDB list

chains:

hotspots:

lengths:

num designs:

Save Changes

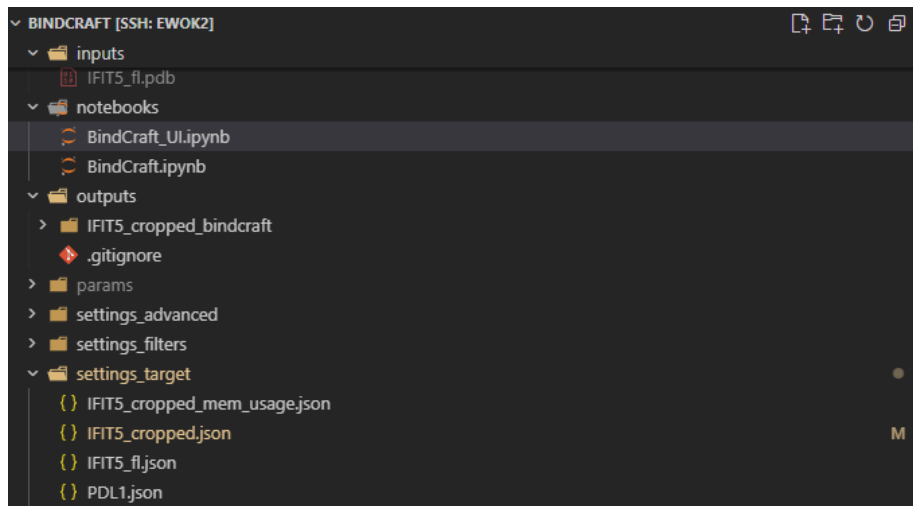
After completing the fields, click **Save Changes**.

Alternatively, upload an existing target settings JSON file that corresponds to the uploaded target PDB, rather than creating a new configuration manually.

Step 2: If not already present, upload a JSON file to /storage/viet/BindCraft/settings_target/

Upload JSON (0)

Both uploaded and newly created target settings files are saved in the project's settings_target/ directory.



Step 3-6: Run BindCraft on specific target setting

Target setting selection

From the Target JSON dropdown menu, select the target settings file that you created or uploaded.

Select a target JSON file (refresh after upload/create):

Target JSON: IFIT5_cropped.json

Selected Target JSON: IFIT5_cropped.json

Refresh JSON

IFIT5_cropped_mem_usage.json

IFIT5_fl.json

PDL1.json

Configure the GPU

Select the GPU to use for the BindCraft job.

BindCraft options

GPU: GPU 0: NVIDIA GeForce RTX 4080 SUPER — 16078 MiB free / 16376 MiB

Auto (default visible GPUs)

GPU 0: NVIDIA GeForce RTX 4080 SUPER — 16078 MiB free / 16376 MiB

GPU 1: NVIDIA GeForce RTX 4080 SUPER — 16075 MiB free / 16376 MiB

Track GPU memory

Off by default. When e

Review the **nvidia-smi** panel for information about the available GPUs and their current usage.

nvidia-smi details for the current selection

How to read **nvidia-smi**: The report lists every GPU visible on this machine. **Memory-Usage** is current VRAM use / capacity; **GPU-Util** shows how busy the GPU is: 0% means it is mostly idle, while 100% means it is very busy. This is separate from memory usage. **Pwr-Usage/Cap** is current power draw / power limit, and **Perf** is the performance state (P0 is highest performance). The **Processes** section shows programs currently using GPU memory.

```

Mon Sep / 19:21:16 2026
+-----+
| NVIDIA-SMI 535.274.02                Driver Version: 535.274.02   CUDA Version: 12.2   |
+-----+-----+-----+-----+-----+-----+
| GPU  Name      Persistence-M   Bus-Id        Disp.A    Volatile Uncorr. ECC  |
| Fan  Temp     Perf          Pwr:Usage/Cap       Memory-Usage  GPU-Util  Compute M.  |
|                                           Memory-Usage  MIG M.     |
+-----+-----+-----+-----+-----+-----+
| 0   NVIDIA GeForce RTX 4080 ...    Off      00000000:08:00:00 Off      0%         Default  N/A  |
| 0%   46C      P8              8W / 320W           1MiB / 16376MiB             N/A  |
+-----+-----+-----+-----+-----+-----+
| 1   NVIDIA GeForce RTX 4080 ...    Off      00000000:41:00:00 Off      0%         Default  N/A  |
| 0%   40C      P8             11W / 320W           1MiB / 16376MiB             N/A  |
+-----+-----+-----+-----+-----+-----+

Processes:
+-----+-----+-----+-----+-----+-----+
| GPU  GI  CI       PID   Type   Process name                      GPU Memory |
| ID   ID  ID                                   Usage     |
+-----+-----+-----+-----+-----+-----+
| No running processes found |
+-----+-----+-----+-----+-----+-----+

```

Binder generation against large target proteins can require substantial GPU memory. Enable **Track GPU memory usage** to monitor memory consumption and assess it against the selected GPU's capacity.

☒ **Track GPU memory usage**

Off by default. When enabled, BindCraft writes **gpu_memory_stats.csv** in the design folder.

The **Active BindCraft jobs using GPU memory** panel displays BindCraft jobs currently using GPU resources.

Active BindCraft jobs using GPU memory

Active BindCraft jobs: This table shows only BindCraft processes currently using GPU memory. **GPU** is the GPU number, **PID** identifies the running process, **User** is the account running it, **Process** is the program name, **CWD** shows the project folder it is running from, and **GPU Memory** shows how much memory that job is using. If the table is empty, no BindCraft job is currently using a GPU.

GPU	PID	USER	PROCESS	CWD	GPU MEMORY
0	2429884	viet	python	/storage/viet/BindCraft	12312 MiB

Click **Refresh GPU list** to update the GPU information and active-job displays.

BindCraft options

GPU: GPU 0: NVIDIA GeForce RTX 4080 SUPER — 15820 MiB free / 16376 MiB

☒ **Track GPU memory usage**

Off by default. When enabled, BindCraft writes **gpu_memory_stats.csv** in the design folder.

[Refresh GPU list](#)

Configure binder filtering and design validation

The **Filters** setting defines the quality criteria that binder designs must satisfy to be accepted. These criteria include confidence scores, interface quality, structural clashes, and binding energy.

The **Advanced** setting controls how BindCraft generates and validates binders, including the design algorithm and the AlphaFold2 and ProteinMPNN settings.

Select a **settings_filters** file and a **settings_advanced** file, then generate the run script.

Filters:

Advanced:

Generate BindCraft Run Script with Settings

For the default configuration, select default_filters.json under **Filters** and default_4stage_multimer.json under **Advanced**.

For further information, refer to the following resources:

- **Filter settings:** <https://github.com/martinpacesa/BindCraft#filters>
- **Advanced settings:** <https://github.com/martinpacesa/BindCraft#advanced-settings>
- **BindCraft paper:** <https://doi.org/10.1101/2024.09.30.615802>

Generate the run script and launch the job

Once all settings have been configured, click **Generate BindCraft Run Script with Settings**.

Generate BindCraft Run Script with Settings

Generated command:

```

CUDA_VISIBLE_DEVICES=0
GPU_memory_tracking=disabled
python \
-u \
/storage/viet/BindCraft/bindcraft.py \
--settings \
/storage/viet/BindCraft/settings_target/IFIT5_cropped.json \
--filters \
/storage/viet/BindCraft/settings_filters/default_filters.json \
--advanced \

```

Then click **Run BindCraft** to launch the job using the generated script.

Run BindCraft

```

Started tmux session: IFIT5_cropped_bindcraft
CUDA_VISIBLE_DEVICES=0
python -u /storage/viet/BindCraft/bindcraft.py --settings /storage/viet/BindCraft/settings_target/IFIT5_cropped.json --filters /storage/viet/BindCraft/settings_filters
Design path: /storage/viet/BindCraft/outputs/IFIT5_cropped_bindcraft
Target accepted designs: 1
Log: /storage/viet/BindCraft/outputs/IFIT5_cropped_bindcraft/bindcraft_20260907_195629.log
Attach: tmux attach -t IFIT5_cropped_bindcraft
Kill: tmux kill-session -t IFIT5_cropped_bindcraft

Started tmux session: IFIT5_cropped_bindcraft
CUDA_VISIBLE_DEVICES=0
python -u /storage/viet/BindCraft/bindcraft.py --settings /storage/viet/BindCraft/settings_target/IFIT5_cropped.json --filters /storage/viet/BindCraft/settings_filters
Design path: /storage/viet/BindCraft/outputs/IFIT5_cropped_bindcraft
Target accepted designs: 1
Log: /storage/viet/BindCraft/outputs/IFIT5_cropped_bindcraft/bindcraft_20260907_195629.log
Attach: tmux attach -t IFIT5_cropped_bindcraft
Kill: tmux kill-session -t IFIT5_cropped_bindcraft
  
```

Monitor and manage jobs

Monitor the selected job's progress in the status section at the bottom of the user interface.

Select a running job below to watch its **progress bar**, then click **Abort a running job** if you need to stop it. Use **Refresh Jobs Dropdown** if it is missing.

Select a job: IFIT5_cropped_bindcraft

Accepted:

IFIT5_cropped_bindcraft · 0/1 accepted designs · trajectories: 0 · current: **starting**

Abort a running job

↻ Refresh Jobs Dropdown

Multiple BindCraft jobs can run concurrently. Assign a different **design_path** to each job so that their outputs are stored in separate directories.

Active BindCraft jobs using GPU memory

Active BindCraft jobs: This table shows only BindCraft processes currently using GPU memory. GPU is the GPU number, PID identifies the running process, User is the account running it, Process is the program name, CWD shows the project folder it is running from, and GPU Memory shows how much memory that job is using. If the table is empty, no BindCraft job is currently using a GPU.

GPU	PID	USER	PROCESS	CWD	GPU MEMORY
0	2434401	viet	python	/storage/viet/BindCraft	12466 MiB
1	2435684	viet	python	/storage/viet/BindCraft	12310 MiB

To stop a job, select it from the job dropdown menu and click **Abort a running job**. If a running job is not listed, click **Refresh Jobs Dropdown** to update the available selections.

Select a job: PDL1_bindcraft

Accepted:

PDL1_bindcraft · 0/1 accepted designs · trajectories: 0 · current: **starting in tmux**

Abort a running job

↻ Refresh Jobs Dropdown

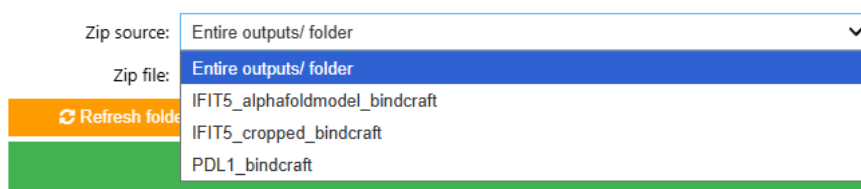
Aborted job 'PDL1_bindcraft' (tmux session 'PDL1_bindcraft').

Step 7: Compress and Download the Outputs

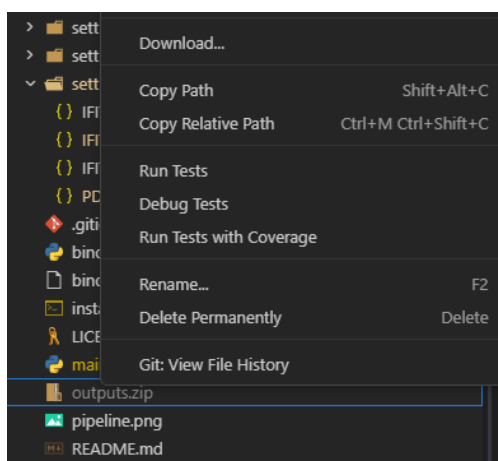
BindCraft output files can be large. Compress the required outputs into a ZIP archive before downloading them.

In the **Zip source** dropdown menu, select either the entire outputs/ directory or an individual design output folder, then create the archive.

Choose the entire **outputs/** folder or a single design folder. The archive is written to the project root (parent of **notebooks/**): **/storage/viet/BindCraft**.



The ZIP archive will be saved in the BindCraft project's root directory. To download it, locate the .zip file in the VS Code **Explorer** panel, right-click it, and select **Download....** Choose a destination on your local computer and save the file.



3.3 Licensing

Licensing

BindCraft is MIT-licensed ([BindCraft](#)) and uses PyRosetta, which is free for academic and non-profit use but requires a commercial license for non-academic use (see [PyRosetta licensing](#) and the [Rosetta Commons licensing FAQ](#)). Ensure your institution has the appropriate licenses before commercial or for-profit use.

[BindCraft](#) uses PyRosetta, which is available for academic and non-profit use, while **commercial or other non-academic use may require a paid license**. For more details, see:

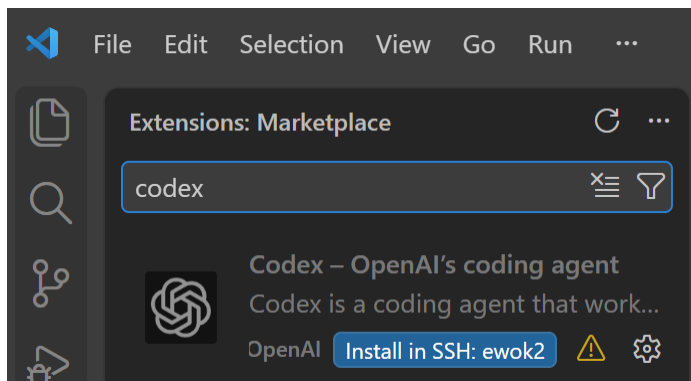
- **PyRosetta licensing:** <https://www.pyrosetta.org/home/licensing-pyrosetta>
- **Rosetta Commons licensing FAQ:** <https://rosettacommons.org/software/licensing-faq/>

4. Useful Extensions

Codex

If you have a ChatGPT account, you can optionally install the Codex extension on the Ewok host for AI-assisted work within the project.

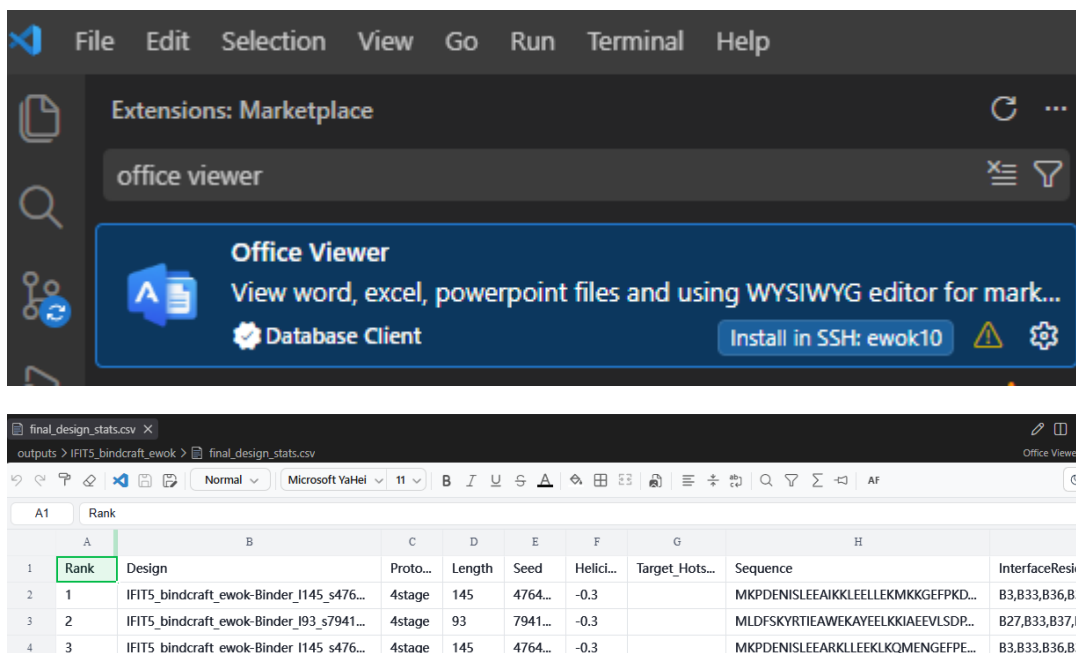
Extension: <https://marketplace.cursorapi.com/items/?itemName=openai.chatgpt>



Office Viewer

This extension helps with reading BindCraft output csv files directly in VS Code.

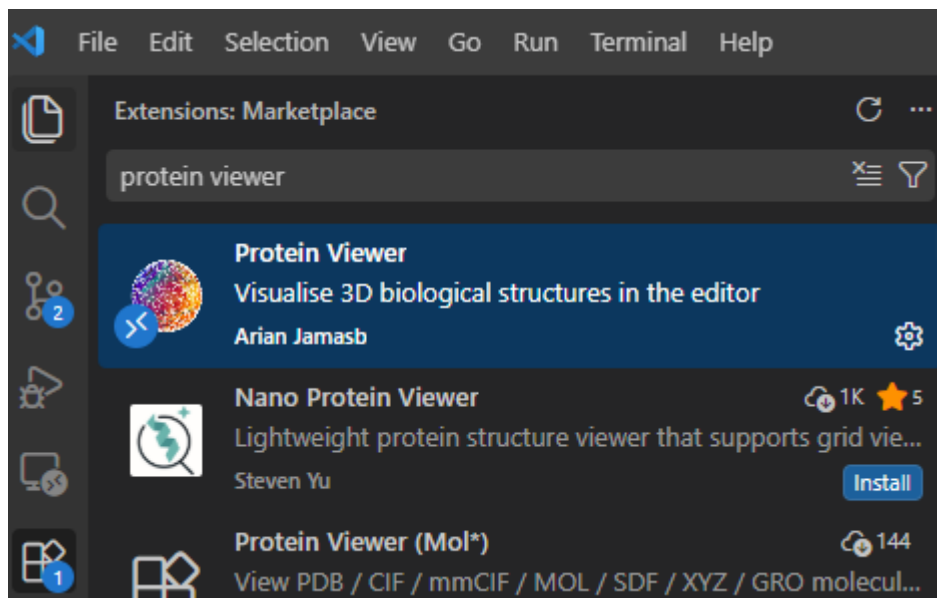
Extension link: <https://marketplace.cursorapi.com/items/?itemName=cweijan.vscode-office>



Protein Viewer

This allows inspection of target protein or generated binders PDB directly in VS Code.

Extension link: <https://marketplace.visualstudio.com/items?itemName=ArianJamasp.protein-viewer>



Right click on a PDB and select *Launch Protein Viewer* to view its 3D structure.

